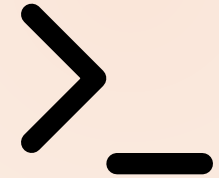


MODULE 01 · 20 MIN

Setup & AI-First Mindset

You + Claude Code = a junior engineer you direct, review, and merge.

Instructor: **Luca Berton** · Endorsed by **Packt Certification**



Welcome · What you'll build today

10 small, real projects in 4 hours — one per module, all with Claude Code.

- **Format:** short theory → live demo → you build it → quick review. Every module.
- **Proof of work:** each module drops a `module-NN/` folder into your final submission zip.
- **Certificate:** pass the quiz + practical + reflection ($\geq 70\%$) → Packt-endorsed certificate.
- **The through-line:** one repeatable loop you take back to your day job on Monday.

You direct. Claude implements. You review and merge. **You are always the engineer of record.**

Your instructor — Luca Berton

- **Automation engineer & educator.** 15+ years shipping infrastructure-as-code, Ansible, and developer tooling for global enterprises.
- **Author & speaker.** Books on Ansible and DevOps; regular conference speaker; runs a YouTube channel on automation.
- **AI-paired delivery practitioner.** Uses Claude Code daily to plan, refactor, document, and review production code.
- **Today's mission:** get you shipping with Claude Code the same way — safely, repeatably, with a loop you can take home.

 lucaberton.com

Anthropic & the Claude models (May 2026)

Anthropic builds Claude — frontier models with a safety-first focus. **Claude Code** is their agentic coding tool.

Model	Best for	Trade-off
Claude Opus	Hardest reasoning: architecture, gnarly refactors, multi-file plans	Slowest · highest cost
Claude Sonnet	The daily driver: most coding, reviews, tests	Balanced speed / cost / quality
Claude Haiku	Fast, cheap: quick edits, summaries, high-volume calls	Less depth on hard problems

Rule of thumb: start on **Sonnet**. Escalate to **Opus** when stuck on design. Drop to **Haiku** for bulk/trivial work. Switch live with `/model` .

Theory · The AI coding loop (3 min)

You stay the engineer of record. Claude proposes; you decide.

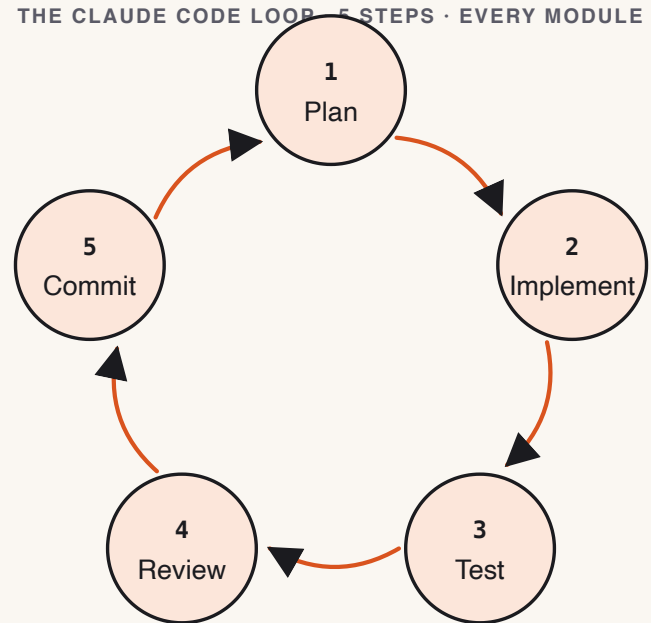
Every module today repeats the same 5 steps:

Plan → Implement → Test → Review → Commit

- **Plan** — write the prompt the way a Tech Lead writes a spec.
- **Implement** — let Claude generate; you read every line.
- **Test** — run it. If it doesn't run, you have nothing.
- **Review** — read it as if it came from a stranger's PR.
- **Commit** — atomic commits, written prose, no `Co-authored-by: Claude`.

Skipping `Review` is the #1 way AI-generated bugs reach production.

The loop you'll repeat all day



Plan → Implement → Test → Review → Commit. Skipping **Review** is how AI bugs ship.

Reference · Claude Code is everywhere (May 2026)

Claude Code ships on **four surfaces** with one shared context:

- **Terminal** — hands-on repo work, `claude -p` piping.
- **VS Code / JetBrains** — inline diffs, gutter actions.
- **Desktop app** — visual diff review, screenshots.
- **Web** — remote/cloud tasks, parallel work, shared sessions.

Today we work in **terminal + IDE**. Patterns transfer to the other surfaces unchanged.

Reference · Slash commands cheat sheet

Command	What it does
/help	List every available slash command
/init	Scaffold a <code>CLAUDE.md</code> for the current repo
/clear	Reset the conversation (forget context)
/compact	Compress history (keeps a summary, saves tokens)
/model	Switch model: Sonnet / Opus / Haiku
/cost	Show token spend and session cost
/review	Review the working-tree diff
/agents · /mcp · /hooks	Manage subagents · MCP servers · hooks
/memory	Open the memory editor
/permissions	Allow / deny tools per project
/doctor	Diagnose env, auth, and integrations
/exit	Leave the session (Ctrl-D works too)

Reference · Common mistakes

- Copying Claude's reply verbatim — the rubric penalises this.
- Treating Review as optional.
- Using PowerShell on Windows — move to WSL2 (see `student-guide.md`).

Live demo · "Read this repo" (5 min)

Watch. Don't type yet.

1. Open this repo in your IDE; run `git status` (clean) + `python3 --version` / `node --version` (green).
2. Paste the prompt verbatim:

```
List the top-level files and tell me what kind of repository this is.
```

3. Claude reads the tree → narrates "workshop repo: slides + exercises + skills".
4. While it responds, narrate the **5-step loop** out loud.

Success signal: Claude names `slides/`, `exercises/`, and `skills/` without you opening them.

▶ Your turn · Verify + name the loop (8 min)

Exercise: `exercises/part-01/README.md`

Step 1 — capture your environment:

```
mkdir -p module-01  
{ python3 --version; node --version; git --version; } > module-01/environment.txt
```

Step 2 — paste into Claude Code, then **rewrite the reply in your own words** into `module-01/loop-notes.md`:

```
In one short paragraph (≤ 6 sentences), explain the loop:  
Plan → Implement → Test → Review → Commit.  
End with one sentence on why skipping Review is the most common failure mode.
```

Success signal: `module-01/` contains both files; the notes name all 5 steps in order.

✓ Done & next (1 min)

Definition of done

- ☐ [] `module-01/environment.txt` — three version strings.
- ☐ [] `module-01/loop-notes.md` — names all 5 steps, **in your own words**.

Next — we apply step 1 (**Plan**) by writing prompts a Tech Lead would sign off on.

Module 2 — Prompting Like a Tech Lead.